



DOCUMENTATION TECHNIQUE

Statistiques DIRM Méditerranée

Préparer le déploiement d'une application sécurisée

Parcours Studi — Blocs 1.1, 1.2 et 1.3

DÉPÔT GITHUB github.com/Meriem1403/Hublot
CI / ACTIONS github.com/Meriem1403/Hublot/actions
APPLICATION (PROD) dirmhublot.netlify.app

Documentation rédigée en janvier 2026

Sommaire

Parcours 1.1 · 1.2 · 1.3

Accès GitHub et Netlify	4
Les bases de la démarche DevOps	5
1.1.1 Les méthodes Agile pour le développement logiciel	5
1.1.2 La démarche DevOps	6
1.1.2.1 Monitoring et supervision	7
1.1.3 Les bases d'un environnement de test	8
1.1.3.1 Guide Docker	9
1.1.3.2 Compléments Docker	10
1.1.4 La mise en place de l'intégration continue (CI)	11
1.1.5 La mise en place de la livraison ou déploiement continu (CD)	12
1.1.5.1 Hébergement de l'application	13
1.1.6 Introduction au YAML	14
Préparer le déploiement d'une application	16
1.2.1 Les enjeux des plans de test	16
1.2.2 Elaborer un scénario de test	17
1.2.3 Mettre en place un environnement de test	18
1.2.4 Les outils et les stratégies des tests de sécurité	19
1.2.4.1 Mesures de sécurité applicative	21
1.2.4.2 Sécurité du déploiement	22
1.2.4.3 Audit des dépendances	23
1.2.4.4 Checklist sécurité	24
1.2.5 Planifier efficacement les tests	26
1.2.6 Valider les résultats des tests	27
1.2.7 Documenter le processus de déploiement	29
1.2.8 Procédure d'exécution des tests	30
1.2.9 Rapport d'exécution des tests	31
1.2.10 Guide Playwright	32
Rédiger des scripts dans la démarche DevOps	35
1.3.1 Les bases du déploiement automatique	35
1.3.2 Rédiger et utiliser un script de déploiement	36
1.3.3 Les bases des scripts d'évolution	37
1.3.3.1 Modèle de données cible	38
1.3.4 Rédiger des scripts d'évolution	39

1.3.4.1 Publier et vérifier les données sur Netlify	40
1.3.5 Optimiser les scripts d'évolution	41
1.3.6 Ecrire un script YAML d'Intégration Continue	42
1.3.7 Automatiser les tests en DevOps	43

Accès au projet

GitHub · Netlify · Connexion

Références du projet : dépôt source, pipeline CI et application déployée.

DÉPÔT GITHUB	https://github.com/Meriem1403/Hublot
WORKFLOWS CI	https://github.com/Meriem1403/Hublot/actions
APPLICATION (PRODUCTION)	https://dirmhublot.netlify.app
PRÉPRODUCTION (STAGING)	https://staging--dirmhublot.netlify.app
CONNEXION AU SITE	Connexion sur le site : identifiants définis dans Netlify (variables VITE_APP_USERNAME et VITE_APP_PASSWORD, non versionnées dans Git).
CONNEXION LOCALE	Développement local (npm run dev) : admin / demo — fichier .env.development

Les bases de la démarche DevOps

1.1.1 Les méthodes Agile pour le développement logiciel

EN BREF

Hublot est développé selon une démarche **Agile pragmatique** : chaque évolution part d'un **besoin métier RH** (user story), se valide par des **critères d'acceptation testables**, et n'est livrée qu'une fois la **Definition of Done** remplie (CI verte, tests documentés). Ce chapitre explique ce cadre, ce qui existe dans le dépôt, et comment il se relie à la chaîne DevOps (document 1.1.2).

Ce qui a été mis en place

Nous n'avons pas mis en place un framework Scrum complet (pas de sprint calendarisé ni de rôles PO/SM formalisés), mais **les artefacts Agile essentiels** sont réels et utilisables :

1. **Modèle de user story** dans GitHub (`.github/ISSUE_TEMPLATE/user-story.yml`) : contexte, story, critères Given/When/Then, checklist DoD.
2. **Scénarios de test ST-F*** et **ST-SEC*** dans le document 1.2.2 : chaque scénario relie un besoin à une preuve d'exécution.
3. **Definition of Done** opérationnelle dans les documents 1.2.5 et 1.2.6 : CI verte, statuts Passé/Échec, rapport daté.
4. **Boucle de feedback automatique** : à chaque push, le workflow **CI** exécute lint, 48 tests unitaires, audit npm, build et E2E — une régression est détectée en quelques minutes.
5. **Priorisation par risque métier** : sécurité d'accès, fiabilité des données RH, lisibilité des tableaux de bord.

Preuves et démonstration

Créer une user story (GitHub) : dépôt → Issues → New issue → template User Story.

Vérifier la DoD localement avant merge :

TERMINAL

```
npm run lint
npm run test:run # 48 tests Vitest
npm run audit:prod
npm run build
npm run test:e2e
```

Check-list opérationnelle (avant merge) :

- Besoin reformulé en user story claire
- Critères d'acceptation ajoutés ou mis à jour (scénarios ST-*)
- CI verte sur la branche
- Statuts mis à jour dans le plan de test (1.3.7)
- Rapport ou preuve d'exécution disponible si campagne manuelle

SYNTHÈSE

« Sur Hublot, l'Agile n'est pas théorique : chaque évolution est formulée en **user story** avec **critères d'acceptation** testables (ST-F* / ST-SEC*), et rien n'est considéré terminé sans **Definition of Done** — dont une **CI verte** à chaque intégration. Cela aligne les livraisons fréquentes (DevOps) sur la **valeur métier RH** et la **qualité mesurable**. »

1.1.2 La démarche DevOps

EN BREF

La **DevOps** sur Hublot, c'est faire circuler le code du poste développeur jusqu'à la **production Netlify** (et optionnellement le **NAS Synology**) de façon **automatisée, contrôlée et documentée**. Le développeur pousse sur Git ; la **CI** vérifie la qualité ; la **CD** déploie seulement si les checks passent. Ce chapitre décrit cette chaîne réelle, pourquoi elle est structurée ainsi, et comment la reproduire.

Ce qui a été mis en place

Chaîne principale

CODE
Développeur (commit/push) → GitHub (dépôt Meriem1403/Hublot) → Workflow CI (.github/workflows/ci.yml) lint → 48 tests → audit npm → build → E2E Playwright → Si CI verte + branche main ## ... (voir dépôt)

Workflows GitHub Actions

FICHIER	NOM DANS GITHUB	RÔLE
ci.yml	CI	Qualité bloquante à chaque push/PR
cd-netlify.yml	CD Netlify	Hook optionnel post-validation
cd-nas.yml	CD NAS Synology	Déploiement SSH/rsync (semi-automatique)
audit-scheduled.yml	Audit npm planifié	Scan hebdomadaire des dépendances
codeql.yml	CodeQL	Analyse statique sécurité (SAST)
security-scan.yml	Security scan	Gitleaks + Trivy

Pratiques DevOps actives

PRATIQUE	ÉTAT	DÉTAIL
Versionnement Git	En place	Historique, branches <code>main</code> / <code>staging</code>
CI qualité	En place	48 tests Vitest, ESLint, audit prod, build, E2E
CD cloud	En place	Netlify, déploiement après checks requis
Staging	En place	Branche <code>staging</code> + previews (voir 1.2.3)
CD NAS	Semi-auto	<code>scripts/deploy-nas.sh</code> + secrets SSH
Monitoring	En place	<code>/health.json</code> , Sentry optionnel (voir 1.1.2.1)
IaC	Périmètre défini	<code>netlify.toml</code> , Docker, workflows
Terraform / Kubernetes	Hors scope	Front statique, pas de cluster

Preuves et démonstration

Voir la CI en action : github.com/Meriem1403/Hublot/actions (<https://github.com/Meriem1403/Hublot/actions>) → workflow **CI** → jobs lint, tests, audit, build, e2e.

Vérifier la prod après deploy :

TERMINAL

```
curl -s https://dirmhublot.netlify.app/health.json  
npm run security:headers https://dirmhublot.netlify.app
```

SYNTHÈSE

« Hublot applique une DevOps **documentée et vérifiable** : le code passe par une **CI bloquante** (lint, 48 tests, audit, build, E2E) avant tout déploiement **Netlify**. Développement et exploitation ne sont pas silos : la même chaîne tourne en local et sur GitHub, avec traçabilité dans Actions et Netlify Deploys. »

1.1.2.1 Monitoring et supervision

EN BREF

Le **monitoring** sur Hublot permet de **détecter les problèmes avant les utilisateurs** : site indisponible, build cassé, erreurs JavaScript en production. Nous combinons une **sonde de disponibilité** (`/health.json`), des **alertes CI/CD** (GitHub, Netlify), des **tests E2E** en filet de sécurité, et **Sentry** (optionnel) pour les erreurs applicatives. Ce chapitre explique chaque brique, pourquoi elle existe, et comment l'activer.

Ce qui a été mis en place

1. Sonde de disponibilité — `public/health.json`

Un fichier JSON statique est servi à la racine du site. Il ne contient **aucune donnée RH** ; il indique seulement que le front est en ligne.

URL production : `https://dirmhublot.netlify.app/health.json`

Contenu attendu :

JSON

```
{  
  "status": "ok",  
  "app": "hublot",  
  "service": "statdirm-frontend"  
}
```

Pourquoi un fichier statique ? Netlify sert les fichiers du dossier `public/` sans logique serveur. C'est simple, fiable, et compatible avec des outils externes (UptimeRobot, curl, scripts de supervision).

2. Erreurs applicatives — Sentry (optionnel)

Le module `src/monitoring/sentry.ts` est chargé depuis `main.tsx` **uniquement** si la variable `VITE_SENTRY_DSN` est définie. Sans DSN, **aucun appel réseau** Sentry n'est effectué (zéro impact performance).

Activation :

1. Créer un projet React sur sentry.io (<https://sentry.io>)
2. Netlify → Site settings → Environment variables :

- `VITE_SENTRY_DSN` = DSN du projet
- `VITE_APP_ENV` = `production` | `staging` | `preview`

1. Redéployer le site

3. Signaux CI / CD

SIGNAL	OÙ LE VOIR	SIGNIFICATION
Échec lint / tests / E2E	GitHub Actions → workflow CI	Code ou régression détectée avant prod
Échec audit npm	Job Audit npm	Dépendance vulnérable
Échec deploy	Netlify → Deploys → logs	Build ou config Netlify incorrecte
NAS hors ligne	Sonde sur <code>http://IP_NAS:8080/health.json</code>	Hébergement interne indisponible

GitHub envoie un **email** en cas d'échec de workflow si les notifications sont activées.

4. Tests E2E comme filet

Les scénarios Playwright (`e2e/smoke.spec.ts`) vérifient en CI :

- Affichage de la page de connexion
- Parcours login → tableau de bord
- Réponse de `/health.json`

Ils complètent le monitoring externe : même si `/health.json` répond, un bug d'authentification serait détecté par l'E2E avant merge.

SYNTHÈSE

« Le monitoring de Hublot repose sur la **sonde** `/health.json`, les **alertes CI/CD** (GitHub, Netlify) et **Sentry** en option pour les erreurs navigateur. L'objectif n'est pas d'attendre qu'un agent signale un dysfonctionnement, mais d'être **alerté dès qu'un contrôle échoue** — disponibilité, build ou parcours critique login. »

1.1.3 Les bases d'un environnement de test**EN BREF**

Un **environnement de test** n'est pas qu'une URL : c'est un **contexte complet** (configuration, données, objectif) dans lequel on exécute des vérifications **sans risquer la production**. Sur Hublot, nous distinguons **cinq niveaux** — DEV, TEST (CI), TEST local, STAGING et PROD — chacun avec un rôle précis dans le cycle de livraison.

Ce qui a été mis en place**Vue d'ensemble**

CODE
DEV (localhost:3000) → push Git


```
TEST / CI (GitHub Actions — VM éphémère)
→ merge staging
STAGING (Netlify, branche staging)
## ... (voir dépôt)
```

Les cinq environnements

ENV.	RÔLE	OÙ	URL / ACCÈS	CONFIGURATION
DEV	Coder, déboguer	Poste local	http://localhost:3000	<code>.env.development</code>
TEST	Qualité automatisée	GitHub Actions — workflow CI	Actions du dépôt	<code>.github/workflows/ci.yml</code>
TEST local	QA « comme la prod »	Docker ou preview Vite	http://localhost:4173	<code>.env.test</code> , <code>docker-compose.test.yml</code>
STAGING	Préproduction métier	Netlify — branche <code>staging</code>	URL Netlify staging	<code>netlify.toml</code> + variables Netlify
PROD	Utilisateurs habilités	Netlify — branche <code>main</code>	https://dirmhublot.netlify.app	Variables Netlify (secrets)

Éléments visibles dans l'application

Le fichier `src/config/environment.ts` lit `VITE_APP_ENV` et affiche un **badge** en DEV et STAGING (« Développement », « Préproduction »), **absent en PROD** pour ne pas perturber les utilisateurs.

Preuves et démonstration

- 1 **DEV** — `npm run dev` → badge « Développement » + login démo.
- 2 **TEST** — GitHub Actions → workflow **CI** → tous les jobs verts.
- 3 **TEST local** — `npm run check:env` ou preview sur :4173.
- 4 **STAGING** — branche `staging` + URL Netlify (si branch deploy activé).
- 5 **PROD** — <https://dirmhublot.netlify.app> — pas de badge, comptes réels.

SYNTHÈSE

« Hublot distingue **cinq environnements** avec des rôles clairs : DEV pour coder, **CI** pour la qualité automatique identique pour tous, TEST local pour le build statique, STAGING pour la validation métier, PROD pour les agents DIRM. Chaque niveau a sa **configuration** (`VITE_APP_ENV`, variables Netlify) et on ne progresse vers la prod qu'avec une **CI verte**. »

1.1.3.1 Guide Docker

EN BREF

Docker permet de lancer Hublot dans un **environnement reproductible** : développement avec rechargement à chaud, **preview de production** sur le port 8080, ou **test local** sur le port 4173. Le projet utilise un **Dockerfile multi-stage** (build Node, service Nginx) et plusieurs fichiers `docker-compose` selon l'usage.

Ce qui a été mis en place

FICHIER	RÔLE
Dockerfile	Stages <code>builder</code> , <code>production</code> (Nginx), <code>development</code> (Vite)
<code>docker-compose.dev.yml</code>	Service dev — http://localhost:3000
<code>docker-compose.prod.yml</code>	Service prod — http://localhost:8080
<code>docker-compose.test.yml</code>	Preview test — http://localhost:4173
<code>docker-compose.yml</code>	Déploiement NAS (montage <code>build/</code>)
Scripts npm	<code>docker:dev</code> , <code>docker:prod</code> , <code>docker:prod:build</code> , <code>test:preview:docker</code>

Preuves et démonstration

TERMINAL

```
docker compose -f docker-compose.dev.yml ps
docker compose -f docker-compose.prod.yml logs -f
npm run docker:stop
```

SYNTHÈSE

« Docker sur Hublot reproduit **dev**, **prod Nginx** et **preview test** via des compose dédiés. C'est l'outil qui rapproche le poste développeur du **NAS** et qui isole la **conversion Excel** sans imposer la même chaîne que Netlify en cloud. »

1.1.3.2 Compléments Docker

EN BREF

Ce document est la **fiche mémo** des commandes Docker les plus utilisées sur Hublot. Le détail (architecture des stages, NAS, dépannage) est dans le Guide Docker.

Commandes essentielles

Développement

TERMINAL

```
npm run docker:dev
## Application : http://localhost:3000

> ### Production locale
bash
npm run docker:prod:build
## Application : http://localhost:8080

> ### Preview test (build statique)
bash
npm run test:preview:docker
## Application : http://localhost:4173

> ### Arrêt et logs
bash
```

```
npm run docker:stop
npm run docker:logs

> ### Évolution des données
bash
bash scripts/run-evolution-pipeline.sh --file "trdata/Suivi_des_emplois_en_ETPT_RPROG (23).xlsx"
```

SYNTHÈSE

« Utilisez les scripts **npm** pour Docker au quotidien ; reportez-vous au **Guide Docker (1.1.3.1)** pour comprendre les fichiers compose et le déploiement NAS. »

1.1.4 La mise en place de l'intégration continue (CI)

EN BREF

L'**intégration continue (CI)** sur Hublot, c'est le workflow GitHub **CI** (`.github/workflows/ci.yml`) : à chaque push ou pull request, la même chaîne s'exécute pour tout le monde — lint, 48 tests, audit npm production, build, E2E Playwright. Si un job échoue, le code n'est pas considéré comme prêt pour la production.

Ce qui a été mis en place

Workflow principal : `ci.yml`

ÉTAPE	COMMANDE	BLO-QUANT
Installation	<code>npm ci</code>	Oui
Lint	<code>npm run lint</code>	Oui
Tests unitaires	<code>npm run test:run</code> (48 tests)	Oui
Audit production	<code>npm run audit:prod</code>	Oui
Build	<code>npm run build</code>	Oui
E2E	<code>npm run test:e2e</code>	Oui

Déclencheurs : push et pull request sur `main` et `staging`.

Workflows complémentaires

FICHIER	RÔLE
<code>codeql.yml</code>	Analyse statique sécurité (SAST)
<code>security-scan.yml</code>	Gitleaks + Trivy
<code>audit-scheduled.yml</code>	Audit npm hebdomadaire

Chaîne logique

CODE
<pre>git push / PR → GitHub Actions (CI)</pre>

```
→ lint → tests → audit → build → E2E
→ si vert : Netlify peut déployer (gate « required checks »)
```

Preuves et démonstration

TERMINAL

```
## Reproduire localement la CI
npm run lint
npm run test:run
npm run audit:prod
npm run build
npm run test:e2e
```

Interface : github.com/Meriem1403/Hublot/actions (<https://github.com/Meriem1403/Hublot/actions>) → workflow CI.

SYNTHÈSE

« La CI Hublot est le fichier `ci.yml` : cinq contrôles bloquants sur chaque push. Elle garantit que le dossier `build/` déployé par Netlify correspond à du code **testé, audité et compilé**. »

1.1.5 La mise en place de la livraison ou déploiement continu (CD)

EN BREF

La livraison continue (CD) publie automatiquement une version validée par la CI. Sur Hublot, la CD cloud passe par **Netlify** (webhook Git + gate sur le workflow CI) ; la CD NAS reste **semi-automatique** via `deploy-nas.sh` et le workflow `cd-nas.yml`.

Ce qui a été mis en place

CD Netlify (principal)

ÉLÉMENT	DÉTAIL
Déclencheur	Push Git (branche connectée, en pratique <code>main</code>)
Config	<code>netlify.toml</code> — <code>npm run build</code> , publish <code>build/</code>
Gate	Deploy only when required checks pass → check CI
URL prod	https://dirmhublot.netlify.app
Staging	Branche <code>staging</code> + deploy previews (voir 1.2.3)

CD NAS (complément)

ÉLÉMENT	DÉTAIL
Script	<code>scripts/deploy-nas.sh</code> (build, tests, rsync optionnel)
Workflow	<code>.github/workflows/cd-nas.yml</code> (<code>workflow_dispatch</code> + secrets SSH)
Runtime	Docker + Nginx sur le Synology (<code>docker-compose.yml</code>)

Workflows associés

FICHIER	RÔLE
<code>ci.yml</code>	Qualité avant toute publication
<code>cd-netlify.yml</code>	Hook optionnel post-CI
<code>cd-nas.yml</code>	Déploiement SSH optionnel

Preuves et démonstration

- Netlify : app.netlify.com (<https://app.netlify.com>) → site → **Deploys**
- GitHub : Actions → **CI** (prérequis) puis éventuellement **CD Netlify**
- Headers prod : `npm run security:headers`

SYNTHÈSE

« La CD Hublot, c'est **Netlify après CI verte** pour le cloud, et un **script NAS** pour l'interne. La CI dit si on peut livrer ; la CD dit où la version est publiée. »

1.1.5.1 Hébergement de l'application

EN BREF

Hublot peut être hébergé de **trois manières** : **Netlify** (production Studi et démo), **Docker/ Nginx** (VPS ou NAS), ou **build manuel + serveur web**. Le canal retenu pour la compétence est **Netlify** ; Docker et Nginx documentent l'option institutionnelle interne.

Ce qui a été mis en place

Production retenue — Netlify

ÉLÉMENT	VALEUR
URL	https://dirmhublot.netlify.app
Config	<code>netlify.toml</code>
Build	<code>npm run build</code>
Publish	<code>build/</code>
Données	<code>public/data/agents.json</code> dans le dépôt ou <code>VITE_APP_DATA_URL</code>
Secrets	Variables Netlify (jamais dans Git)

Alternatives documentées

MÉTHODE	USAGE
<code>docker-compose.prod.yml</code>	VPS ou test local port 80/8080
Build + Nginx	Serveur classique, <code>try_files</code> vers <code>index.html</code>
NAS Synology	<code>docker-compose.yml</code> + script 1.3.2

Preuves et démonstration

- Site live : <https://dirmhublot.netlify.app>
- Config : `netlify.toml` à la racine du dépôt
- Santé : `curl https://dirmhublot.netlify.app/health.json`

SYNTHÈSE

« Hublot est **hébergé en production sur Netlify** (`build/` statique, HTTPS, headers, gate CI). Docker et Nginx restent documentés pour le **NAS** et les serveurs internes, avec les mêmes principes : build Vite, variables au build, SPA sur `index.html` . »

1.1.6 Introduction au YAML

EN BREF

Le **YAML** est le format utilisé pour décrire la **CI** (GitHub Actions), la **CD** (`netlify.toml`), et l'**infrastructure as code** du projet. Sur Hublot, comprendre YAML, c'est lire `ci.yml` et `netlify.toml` : déclencheurs, jobs, commandes, variables et en-têtes HTTP.

Ce qui a été mis en place

FICHIER	RÔLE
<code>.github/workflows/ci.yml</code>	Pipeline CI (lint, tests, audit, build, E2E)
<code>.github/workflows/cd-netlify.yml</code>	Hook Netlify optionnel
<code>.github/workflows/cd-nas.yml</code>	Déploiement NAS optionnel
<code>.github/workflows/codeql.yml</code>	SAST
<code>.github/workflows/security-scan.yml</code>	Gitleaks, Trivy
<code>netlify.toml</code>	Build, publish, contexts, headers, redirects SPA
<code>docker-compose.*.yaml</code>	Services Docker

Preuves et démonstration

TERMINAL

```
## Valider la syntaxe (lecture humaine + PR)
cat .github/workflows/ci.yml
cat netlify.toml
## Après modification : push – onglet Actions – workflow CI
```

SYNTHÈSE

« Sur Hublot, le YAML (et le TOML Netlify) **décrit la CI et la CD** : qui déclenche quoi, quelles commandes `npm` exécuter, où publier `build/`, quels headers appliquer. Savoir lire ces fichiers, c'est savoir **comment le déploiement sécurisé est automatisé**. »

Préparer le déploiement d'une application

1.2.1 Les enjeux des plans de test

EN BREF

Un **plan de test** structure ce qu'il faut vérifier avant de livrer Hublot : données RH, accès au tableau de bord, chaîne DevOps. Sur le dépôt `Meriem1403/Hublot`, nous combinons **48 tests unitaires Vitest**, une **CI bloquante** et des **scénarios documentés** (ST-F*, ST-SEC*) complétés par des contrôles manuels tracés. Ce chapitre explique pourquoi ce cadre est indispensable pour un déploiement sécurisé sur `dirmhublot.netlify.app`.

Ce qui a été mis en place

Nous avons structuré la qualité autour de trois piliers. Le **tableau de plan** (1.3.7 Automatiser les tests en DevOps) liste objectifs, résultats attendus et statuts. Les **scénarios formalisés** (1.2.2) décrivent les cas manuels et sécurité. La **CI** (`.github/workflows/ci.yml`) exécute lint, 48 tests, audit production, build et E2E Playwright à chaque push sur `main` et `staging`.

NIVEAU	OUTIL	VOLUME HUBLLOT
Unitaire	Vitest	48 tests (métier + sécurité)
Intégration	Build Vite + preview	Artefact <code>build/</code>
Système	Playwright	3 smoke + campagne ST-F*
Sécurité	npm audit, Gitleaks, CodeQL, headers	Voir 1.2.4
Acceptation	Tests manuels documentés	ST-F01 à ST-F06

Preuves et démonstration

TERMINAL

```
npm run test:run # 48 tests – base du plan
npm run lint
npm run audit:prod
npm run build
npm run test:e2e
```

PREUVE	OÙ LA TROUVER
Run CI vert	github.com/Meriem1403/Hublot/actions (https://github.com/Meriem1403/Hublot/actions)
Statuts plan	1.3.7
Campagne datée	1.2.9
Commandes de validation	1.2.8.md)

Limites assumées : pas de tests de charge ; couverture code non à 100 % mais ciblée sur le métier ; données RH réelles validées en staging, pas en CI (mocks).

SYNTHÈSE

Le plan de test Hublot structure quoi valider, comment et quand livrer : fiabilité des données RH, sécurité d'accès et absence de régression dans une chaîne DevOps automatisée, prouvée par 48 tests unitaires, CI et scénarios tracés.

1.2.2 Elaborer un scénario de test

EN BREF

Un **scénario de test** décrit un comportement attendu de Hublot dans un cas d'usage identifié, au-delà d'une ligne du plan. Nous utilisons la structure **Étant donné / Quand / Alors** et des identifiants stables **ST-F*** (fonctionnel), **ST-SEC*** (sécurité) et **ST-AUTO*** (automatisation CI). Ce document est le référentiel opérationnel des scénarios ; l'annexe livrable correspond à Annexe 13.

Ce qui a été mis en place

Nous avons catalogué les scénarios fonctionnels **ST-F01** à **ST-F06**, les scénarios sécurité **ST-SEC01** à **ST-SEC05**, et la couverture automatisée **ST-AUTO-01** à **ST-AUTO-05**. Une campagne Playwright (`npm run test:campaign`) re-joue plusieurs ST-F* en QA locale et en lecture seule sur la prod. Le suivi d'exécution est tenu dans 1.2.9.

Correspondance avec le plan de test

ID SCÉNARIO	LIGNE DU TABLEAU 1.3.7
ST-F01	Test authentification
ST-F02	Test chargement des données
ST-F03	Test des filtres
ST-F04	Test responsive
ST-F05	Test performance
ST-F06	Test badge environnement
ST-SEC01	Test sécurité (headers)
ST-SEC02	Garde-fous applicatifs
ST-SEC03	Aucun secret commité (Gitleaks)
ST-SEC04	Dépendances de production (audit + Trivy)
ST-SEC05	Analyse statique (CodeQL)
ST-AUTO-*	Lint, tests unitaires, E2E, build, audit

Preuves et démonstration

TERMINAL

```
npm run test:run # ST-AUTO-02, ST-SEC02 (partie)
npm run test:e2e # ST-AUTO-03, ST-F01 (partie)
npm run test:campaign # ST-F01...F06 + PROD lecture seule
npm run security:headers # ST-SEC01
npm run security:test # ST-SEC02 (15 tests)
```

Les sections suivantes détaillent chaque scénario. Les statuts de campagne sont dans le tableau de suivi en fin de document.

SYNTHÈSE

Les scénarios Hublot formalisent les cas critiques (auth, données, filtres, sécurité HTTP et applicative) avec des identifiants ST-* traçables vers le plan de test et la CI du dépôt `Meriem1403/Hublot`.

1.2.3 Mettre en place un environnement de test

EN BREF

Hublot s'exécute à **quatre niveaux** : développement local, CI GitHub, staging Netlify et production. Ce chapitre décrit comment nous avons configuré la **branche** `staging`, les **Deploy Previews** et les variables d'environnement pour tester sans impacter `dirmhublot.netlify.app`. Il complète les fondamentaux du 1.1.3.

Ce qui a été mis en place

Nous avons défini quatre contextes d'exécution et aligné la configuration Netlify.

NIVEAU	OÙ	URL TYPE
DEV	<code>npm run dev</code>	<code>http://localhost:5173</code>
TEST	GitHub Actions — workflow CI	Onglet Actions du dépôt
STAGING	Branche <code>staging</code> ou preview PR	<code>staging--dirmhublot.netlify.app</code> ou <code>deploy-preview-NN-...</code>
PROD	Branche <code>main</code>	<code>https://dirmhublot.netlify.app</code>

Le fichier `netlify.toml` définit `VITE_APP_ENV = "staging"` pour le contexte staging. La CI s'exécute sur `main` et `staging` avec les mêmes jobs (lint, 48 tests, audit, build, E2E).

Preuves et démonstration

TERMINAL

```
npm run check:env # lint + tests + audit + build en local
npm run build:e2e # artefact QA (e2e-user / e2e-pass)
npm run preview:test # http://127.0.0.1:4173
```

VÉRIFICATION	COMMANDE / ACCÈS
CI sur <code>staging</code>	GitHub Actions → workflow CI
Preview PR	Netlify → Deploy Preview de la PR
Prod	Navigateur → <code>https://dirmhublot.netlify.app</code>

SYNTHÈSE

L'environnement de test Hublot sépare DEV, CI, staging Netlify et production : même pipeline de qualité sur `main` et `staging`, URLs de préproduction pour les scénarios manuels, promotion contrôlée vers `dirmhublot.netlify.app`.

1.2.4 Les outils et les stratégies des tests de sécurité

EN BREF

Les tests de sécurité de Hublot cherchent les failles **avant** la production : données RH, accès au tableau de bord, dépendances npm, secrets Git, en-têtes HTTP. Nous appliquons le **shift-left** (dès la CI) et la **défense en profondeur** (plusieurs familles d'outils). Ce chapitre recense outils, commandes et scénarios ST-SEC* du dépôt `Meriem1403/Hublot`.

Ce qui a été mis en place

Nous croisons six familles de contrôles intégrés au dépôt et à GitHub Actions.

OUTIL	FAMILLE	RÔLE	INTÉGRATION
<code>npm audit / audit:prod</code>	SCA	CVE prod — critical bloquant, high via allowlist	<code>ci.yml</code> + <code>audit-scheduled.yml</code>
Dependabot	SCA préventif	PR de mise à jour npm / Actions	<code>.github/dependabot.yml</code>
Trivy	SCA	Scan complémentaire filesystem	<code>security-scan.yml</code> (informatif)
CodeQL	SAST	Motifs dangereux JS/TS	<code>codeql.yml</code>
Gitleaks	Secrets	Scan arborescence	<code>security-scan.yml</code> (bloquant)
Vitest <code>security.test.ts</code>	Unitaire	Sanitization, filtres, session, masquage RH	CI via <code>test:run</code>
<code>check-security-headers.sh</code>	Configuration	HTTPS + en-têtes (ST-SEC01)	<code>npm run security:headers</code>
Playwright	E2E accès	Login refusé / session	CI <code>test:e2e</code>
ESLint	Qualité préventive	Réduction de motifs à risque	CI <code>lint</code>

Principe : ce qui est certain et maîtrisé **bloque** (audit prod, Gitleaks) ; Trivy et CodeQL **informent** dans l'onglet Security pour triage.

Preuves et démonstration

Ordre conseillé pour une démonstration

#	COMMANDE	SCÉNARIO
1	<code>npm run security:test</code>	ST-SEC02
2	<code>npm run audit:prod</code>	ST-SEC04
3	<code>npm run security:headers</code>	ST-SEC01
4	<code>npm run test:run</code>	ST-AUTO / ST-SEC02 (48 tests)
5	<code>npm run lint</code>	ST-AUTO-01

TERMINAL

```
npm run security:test
npm run audit:prod
npm run security:headers
```

Sur GitHub : **Actions** (CI, CodeQL, Scans sécurité) ; **Security** → Code scanning, Dependabot.

Couverture par scénarios

SCÉNARIO	INTENTION	OUTIL
ST-SEC01	En-têtes HTTP prod	<code>security:headers</code> / curl
ST-SEC02	Garde-fous applicatifs	<code>security.test.ts</code>
ST-SEC03	Pas de secret commité	Gitleaks
ST-SEC04	Dépendances prod	<code>audit:prod</code> + Trivy
ST-SEC05	Faibles code	CodeQL
ST-F01	Accès refusé	Playwright

Détail des scénarios : 1.2.2.

Correspondance OWASP (repère)

CATÉGORIE OWASP	COUVERTURE HUBLOT
A01 Broken Access Control	Auth + E2E login
A02 Cryptographic Failures	HTTPS/TLS Netlify, HSTS
A03 Injection / XSS	<code>sanitizeInput</code> + Vitest + CodeQL
A05 Security Misconfiguration	En-têtes + ST-SEC01
A06 Vulnerable Components	<code>audit:prod</code> , Dependabot, Trivy
A07 Auth Failures	Session + expiration (tests)
Secrets exposure	Gitleaks + variables Netlify

Limites et évolutions

CONSTAT	ÉVOLUTION
CSP absente sur Netlify (HSTS + X-Frame + nosniff présents)	Aligner <code>netlify.toml</code> sur <code>nginx.conf</code>
Auth côté client (<code>VITE_*</code>)	JWT / SSO serveur
Pas de DAST	OWASP ZAP baseline sur staging
CORS <code>*</code> fonction Neon	Restreindre à l'origine front
Trivy informatif	Bloquant après triage

Glossaire outils et menaces (référence)

SIGLE / TERME	EN CLAIR
CVE	Identifiant public d'une vulnérabilité connue
XSS	Injection de script dans une page
Sanitization	Nettoyage des entrées utilisateur
HSTS	Force HTTPS
CSP	Restreint les sources de scripts (XSS)
X-Frame-Options	Anti-clickjacking
nosniff	Anti MIME-sniffing
SARIF	Format des alertes dans l'onglet Security GitHub

SYNTHÈSE

La sécurité Hublot combine shift-left et défense en profondeur : à chaque intégration, SCA, SAST, scan de secrets, tests Vitest et E2E d'accès ; après déploiement, contrôle des en-têtes HTTP. Le bloquant arrête la livraison ; le reste alimente la revue dans GitHub Security.

1.2.4.1 Mesures de sécurité applicative

EN BREF

La sécurité **applicative** de Hublot protège les données RH dans le navigateur : authentification avant le tableau de bord, sanitization des entrées, masquage des champs sensibles, session limitée dans le temps. Ces mesures sont codées dans `src/utils/security.ts` et `AuthGuard`, et vérifiées par **15 tests Vitest** inclus dans les **48 tests** de la CI.

Ce qui a été mis en place

Authentification et autorisation

- **Page de connexion** avant tout accès au tableau de bord.
- **AuthGuard** : vérification au chargement des routes protégées.
- **Session** : `sessionStorage`, expiration testée (8 heures).
- **Identifiants** : `VITE_APP_USERNAME` / `VITE_APP_PASSWORD` (Netlify ou `.env` local, jamais dans Git).

Évolution documentée : auth serveur (JWT / SSO) pour renforcer le modèle actuel côté client.

Validation et sanitization

- `sanitizeInput()` : neutralise balises HTML, `javascript:`, handlers d'événements.
- `validateFilter()` : n'accepte que les valeurs de filtre attendues (région, service, etc.).

Protection des données sensibles

Dans `src/utils/security.ts` :

FONCTION	RÔLE
<code>maskName()</code>	Masque noms / prénoms
<code>maskId()</code>	Masque identifiants
<code>maskDateOfBirth()</code>	N'affiche que l'année de naissance

Les fichiers `agents.json` et exports Excel sont **hors dépôt** (`.gitignore`).

Déploiement NAS (complément)

Le fichier `nginx.conf` prévoit HTTP Basic, rate limiting, blocage d'accès direct aux `.json`, headers CSP/HSTS — référence pour hébergement Synology, distinct de Netlify prod.

Preuves et démonstration

TERMINAL

```
npm run security:test
## Attendu : 15 passed

npm run test:run
## Attendu : 48 passed (dont sécurité)
```

TEST MANUEL	SCÉNARIO
Login invalide → message d'erreur, pas de dashboard	ST-F01
Session expirée après 8 h (logique testée en Vitest)	ST-SEC02

Stratégie globale : 1.2.4. Scénarios : 1.2.2.

Conformité RGPD (rappel) : base légale, information des personnes, mesures techniques — responsabilité institutionnelle DIRM, au-delà du seul code.

SYNTHÈSE

Hublot applique auth, sanitization, validation des filtres et masquage RH côté application, avec 15 tests Vitest dédiés intégrés à la CI sur `Meriem1403/Hublot`.

1.2.4.2 Sécurité du déploiement

EN BREF

La sécurité du **déploiement** Hublot repose sur HTTPS Netlify, variables d'environnement hors Git, exclusion des données RH du dépôt, en-têtes HTTP durcis et audit npm bloquant en CI. L'authentification applicative protège l'accès au tableau de bord sur `https://dirmhublot.netlify.app`. Annexe associée : Annexe 07.

Ce qui a été mis en place

HTTPS

CANAL	MISE EN ŒUVRE
Netlify (PROD)	Certificat TLS automatique
NAS (local)	HTTP par défaut ; HTTPS via reverse proxy DSM + certificat interne

Variables d'environnement et secrets

VARIABLE (EXEMPLES)	OÙ	USAGE
<code>VITE_APP_USERNAME</code>	Netlify	Connexion application
<code>VITE_APP_PASSWORD</code>	Netlify	Connexion application
<code>NETLIFY_DATABASE_URL</code>	Netlify (Neon)	Base PostgreSQL

Local : fichier `.env` (`.gitignore`). Règles : pas de commit de secrets ; rotation en cas de fuite ; données RH hors dépôt public.

Headers de sécurité (`netlify.toml`)

HEADER	VALEUR	PROTECTION
<code>X-Frame-Options</code>	<code>DENY</code>	Clickjacking
<code>X-Content-Type-Options</code>	<code>nosniff</code>	MIME sniffing
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	Fuite de référent
<code>Permissions-Policy</code>	camera/micro/geo désactivés	Permissions navigateur

Audit npm en CI

`npm run audit:prod` est **bloquant** dans `ci.yml` — politique en 1.2.4.3.

Authentification applicative

Page de connexion ; identifiants au build ; session navigateur — tests ST-F01, E2E Playwright.

Preuves et démonstration

TERMINAL	
<pre>curl -I https://dirmhublot.netlify.app npm run audit:prod npm run security:headers</pre>	
PREUVE	ATTENDU
CI job Audit npm	Exit 0
<code>security:headers</code>	Exit 0, X-Frame-Options + nosniff
<code>.gitignore</code>	<code>.env</code> , <code>agents.json</code> , <code>*.xlsx</code> exclus

Checklist complète : 1.2.4.4. Mesures applicatives : 1.2.4.1.

SYNTHÈSE

Le déploiement Hublot est sécurisé par HTTPS, secrets hors Git, headers HTTP, audit npm en CI et authentification avant accès aux tableaux de bord RH.

1.2.4.3 Audit des dépendances

EN BREF

L'**audit npm** vérifie que les librairies embarquées en production n'introduisent pas de CVE critiques non maîtrisées. Sur Hublot, `npm run audit:prod` est **bloquant** en CI ; une allowlist documente les exceptions assumées. Un audit hebdomadaire complète le contrôle à chaque push.

Ce qui a été mis en place

COMMANDE	USAGE
<code>npm run audit:prod</code>	Audit production — utilisé en CI
<code>npm audit</code>	Rapport complet (dev inclus) — diagnostic local

Règles CI

1. **Critical** sur dépendances prod → **bloquant**, sans exception.
2. **High** sur dépendances prod → bloquant sauf packages dans `security/audit-allowlist.json`.
3. Rapport hebdomadaire : workflow **Audit npm planifié** (lundi 6h UTC).

Allowlist actuelle

PACKAGE	MOTIF
<code>react-simple-maps</code> (+ chaîne d3)	Cartographie ; correctif = downgrade majeur ; usage client interne

Fichier : `security/audit-allowlist.json` — toute nouvelle entrée exige motif métier et date de revue en PR.

Scénario lié : **ST-SEC04** (1.2.2).

Preuves et démonstration

TERMINAL	
<pre>npm run audit:prod ## Attendu : exit 0 en l'état du dépôt validé</pre>	
PREUVE	OÙ
Job Audit npm vert	GitHub Actions → workflow CI
Artefact hebdo	Workflow audit planifié
Allowlist versionnée	<code>security/audit-allowlist.json</code>

SYNTHÈSE

L'audit des dépendances Hublot bloque les CVE critical en production et encadre les high via allowlist documentée, exécuté à chaque CI sur `Meriem1403/Hublot`.

1.2.4.4 Checklist sécurité

EN BREF

Cette checklist regroupe les contrôles à effectuer **avant et après** un déploiement de Hublot sur Netlify ou NAS. Elle adapte les exigences Studi au dépôt réel `Meriem1403/Hublot` : pas d'emoji, preuves par commandes (`audit:prod` , `security:headers`) et statuts ST-SEC*.

Ce qui a été mis en place

Nous utilisons cette checklist en complément de la CI (qui automatise une partie des points). Les cases manuelles couvrent Netlify, revue des secrets et tests ST-F01 / ST-SEC01.

Preuves et démonstration

Commandes de validation rapide :

TERMINAL
<pre>npm run test:run && npm run audit:prod && npm run security:headers</pre>

GitHub : dernier run **CI** vert + workflow **Scans sécurité** (Gitleaks).

Checklist détaillée (à cocher)

Authentification

- `VITE_APP_USERNAME` / `VITE_APP_PASSWORD` configurés sur Netlify (prod et staging distincts si possible)
- `.env` absent du dépôt Git
- AuthGuard testé — ST-F01 **Passé**
- Déconnexion efface l'accès au tableau de bord
- Accès sans authentification bloqué (E2E + test manuel)

HTTPS / TLS (Netlify prod)

- URL prod en `https://dirmhublot.netlify.app`
- `curl -I` ou `npm run security:headers` — ST-SEC01 **Passé**
- HSTS présent (`strict-transport-security`)
- (NAS) Certificat et redirection HTTP→HTTPS si déploiement local

Headers de sécurité

- `X-Frame-Options: DENY` (ou équivalent)
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy` configurée (`netlify.toml`)
- `Permissions-Policy` restrictive
- (Évolution) CSP alignée avec `nginx.conf`

Protection des fichiers et données

- `agents.json`, `*.xlsx`, `.env` dans `.gitignore`
- Aucun export RH dans le README public
- Gitleaks sans fuite réelle — ST-SEC03
- (NAS) Accès direct `.json` bloqué (403)

Dépendances et CI

- `npm run audit:prod` exit 0 — ST-SEC04
- CI verte sur le commit déployé (48 tests, lint, build, E2E)
- Allowlist `security/audit-allowlist.json` à jour et justifiée
- Dependabot / revue alertes Security

Tests de sécurité applicative

- `npm run security:test` — 15 passed (ST-SEC02)
- CodeQL consulté — ST-SEC05 (onglet Security)
- Rollback Netlify testé au moins une fois (1.2.7)

Docker / NAS (si applicable)

- Pas de secrets dans l'image
- Healthcheck configuré
- Volumes sensibles en lecture seule

Conformité RGPD (organisationnel)

- Base légale et information des personnes — responsabilité DIRM
- Durée de conservation définie
- Mesures techniques documentées (1.2.4.1)

Post-déploiement

- Application accessible en HTTPS
- Login fonctionnel avec comptes prod
- `/health.json` répond `{ "status": "ok" }`
- Monitoring / Sentry configuré si activé

Date de vérification : _____

Vérifié par : _____

Prochaine révision : _____

SYNTHÈSE

La checklist sécurité Hublot consolide les contrôles auth, transport, headers, dépendances et données avant publication sur Netlify, en s'appuyant sur la CI et les scénarios ST-SEC*.

1.2.5 Planifier efficacement les tests

EN BREF

Planifier efficacement, ce n'est pas « tout tester » : c'est cibler les risques qui coûtent cher s'ils se produisent (données RH, accès, régression), choisir le bon niveau de test et tracer les résultats. Sur Hublot, la méthode en six étapes relie risques, scénarios ST-*, les **48 tests Vitest** et la CI du dépôt `Meriem1403/Hublot`.

Ce qui a été mis en place

Méthode en 6 étapes

Schéma interactif : consulter la version Markdown du dépôt.

ÉTAPE	QUESTION	LIVRABLE
1. Risques	Qu'est-ce qui ferait mal ?	Tableau § risques ci-dessous
2. Priorités	Critique / Haute / Moyenne / Basse ?	1.2.2
3. Type	Auto ou manuel ?	Pyramide + règles § auto vs manuel
4. Scénarios	Cas Étant donné / Quand / Alors ?	ST-F01...ST-SEC05
5. Environnement	DEV, CI, STAGING, PROD ?	1.2.3
6. Traçabilité	Passé / Échec ?	1.3.7 + Actions

Batterie unitaire (48 tests)

FICHER	TESTS	RÔLE
<code>dataCalculations.test.ts</code>	20	Calculs RH (âge, ETP, répartitions)
<code>dataService.test.ts</code>	12	Filtres, normalisation, chargement
<code>security.test.ts</code>	15	Sanitization, session, masquage
<code>environment.test.ts</code>	1	Libellé environnement

Preuves et démonstration

#	COMMANDE	MESSAGE DÉMO
1	<code>npm run test:run</code>	48 tests — base pyramide
2	<code>npm run lint</code>	Qualité statique
3	<code>npm run audit:prod</code>	Dépendances prod
4	<code>npm run build</code>	Artefact déployable
5	<code>npm run test:e2e</code>	Parcours login
6	<code>npm run test:campaign</code>	Scénarios ST-F*
7	<code>npm run check:env</code>	Chaîne locale complète

TERMINAL

```
npm run test:run
npm run lint
npm run audit:prod
npm run build
npm run test:e2e
npm run test:campaign
```

GitHub **Actions** : pipeline CI lint → unit-tests → audit → build → e2e.

SYNTHÈSE

Planifier efficacement sur Hublot, c'est partir des risques RH et d'accès, prioriser, placer 48 tests unitaires en base, la CI au milieu, peu d'E2E au sommet, et tracer chaque résultat dans le plan et les rapports.

1.2.6 Valider les résultats des tests

EN BREF

Valider un résultat, ce n'est pas lancer une commande : c'est comparer le résultat obtenu au **résultat attendu**, statuer **Passé** ou **Échec**, conserver une **preuve**, puis décider si la livraison est acceptable. Sur Hublot, la CI valide l'automatisé ; la campagne Playwright et le rapport daté valident les scénarios ST-F*.

Ce qui a été mis en place

Nous appliquons une procédure en cinq étapes et une matrice de référence alignée sur 1.3.7.

Schéma interactif : consulter la version Markdown du dépôt.

Preuves et démonstration

Matrice de référence (état documenté)

ID / TEST	TYPE	STATUT	PREUVE
ST-AUTO lint	Auto	Passé	CI
ST-AUTO unitaires	Auto	Passé	48 tests
ST-AUTO build	Auto	Passé	CI + Netlify
ST-AUTO audit	Auto	Passé	CI
ST-AUTO E2E	Auto	Passé	CI
ST-F01 ... F06	Manuel/E2E	Passé	Campagne 2026-05-27
ST-SEC01	Semi-auto	Passé	<code>security:headers</code>
ST-SEC02	Auto	Passé	15 tests
ST-SEC03	Auto	À vérifier	security-scan.yml
ST-SEC04	Auto	Passé	CI + Trivy info
ST-SEC05	Auto	À vérifier	CodeQL Security

Checklist de validation

- 1 `npm run test:run` → 48 passed
- 2 GitHub Actions → CI vert
- 3 1.3.7 → colonne Statut
- 4 1.2.9 → date campagne
- 5 `npm run security:headers` → ST-SEC01

TERMINAL

```
npm run test:run && npm run lint && npm run audit:prod && npm run build && npm run security:test
```

Décision de livraison (DoD)

- CI verte sur le commit livré
 - Aucun **Échec** Critique dans le plan
 - ST-F01, ST-F02, ST-F03 **Passé**
 - ST-SEC01 **Passé**
 - Rapport à jour
 - Pas de secret dans le diff
- Un seul Critique en **Échec** → pas de livraison.

SYNTHÈSE

Valider les tests Hublot, c'est comparer, statuer avec preuve, et n'accepter la livraison que si la CI et les scénarios critiques sont Passé sur `Meriem1403/Hublot`.

1.2.7 Documenter le processus de déploiement

EN BREF

Ce chapitre documente comment **installer**, **déployer**, **rollback** et **observer** Hublot : procédure locale, configuration Netlify, architecture Git → CI → CD, captures du pipeline et consultation des logs. Le site de production est `https://dirmhublot.netlify.app` ; le dépôt source est `Meriem1403/Hublot`.

Ce qui a été mis en place

Procédure d'installation locale

Prérequis : Node.js 20 LTS, npm, accès GitHub et Netlify.

TERMINAL

```
git clone https://github.com/Meriem1403/Hublot.git
cd Hublot
npm ci
## .env local (jamais commité) : VITE_APP_USERNAME, VITE_APP_PASSWORD
npm run dev

> Application : `http://localhost:5173`. → Vérification avant déploiement :
bash
npm run test:run # 48 tests
npm run build # génère build/
```

Déploiement Netlify

1. Add new site → importer **Meriem1403/Hublot**, branche **main**.
2. Build command : `npm run build` ; publish directory : `build` (`netlify.toml`).
3. Variables : `VITE_APP_USERNAME`, `VITE_APP_PASSWORD`, `NETLIFY_DATABASE_URL` (Neon).
4. Chaque push `main` déclenche build + deploy → `https://dirmhublot.netlify.app`.

Architecture

CODE

```
Développeur → git push main
→ GitHub (Meriem1403/Hublot)
→ GitHub Actions CI (lint, 48 tests, audit, build, E2E)
→ Netlify CD (npm run build → publish build/)
→ https://dirmhublot.netlify.app
```

ANNEXE	FICHIER	RÔLE
01	<code>ci.yml</code>	Pipeline CI
02	<code>netlify.toml</code>	Build, headers, SPA
03	<code>package.json</code>	Scripts npm
05	Architecture déploiement	Détail schéma

Voir Annexe 05.

Preuves et démonstration

—URL prod : `https://dirmhublot.netlify.app`

—Dernier run CI vert sur le SHA déployé

—Extrait log : fin étape Build `built in ...`
 —Rollback testé au moins une fois (checklist 1.2.4.4)
 CD détaillé : 1.1.5.md).

SYNTHÈSE

Le processus de déploiement Hublot est documenté de `git clone` à la prod Netlify, avec rollback, schéma CI/CD et emplacements des logs pour audit et traçabilité.

1.2.8 Procédure d'exécution des tests

EN BREF

Ce chapitre liste les **commandes et étapes** pour reproduire de manière reproductible les résultats du plan de test Hublot : 48 tests Vitest, build, CI GitHub, contrôles prod, démo Playwright. Annexe associée : Annexe 09.

Ce qui a été mis en place

Ordre d'exécution documenté : unitaires → CI → E2E → headers prod → sécurité / rollback. Scripts npm : `test:run`, `build`, `test:e2e`, `test:e2e:demo`, `security:headers`, `test:campaign`.

Preuves et démonstration

Ordre d'exécution recommandé

#	ACTION	DURÉE
1	<code>npm run test:run</code>	~2 min
2	GitHub Actions CI vert	~1 min
3	<code>npm run test:e2e:demo</code> ou prod lecture seule	~3 min
4	<code>curl -I</code> ou <code>security:headers</code>	~30 s
5	Rollback + 1.2.4.2	~1 min

Validation des statuts : 1.2.6.

SYNTHÈSE

La procédure Hublot enchaîne les mêmes contrôles que la CI, les tests E2E et les vérifications prod, reproductibles depuis le dépôt `Meriem1403/Hublot`.

1.2.9 Rapport d'exécution des tests

EN BREF

Ce rapport synthétise une **campagne de tests** datée du 27 mai 2026 sur Hublot : environnements QA local et PROD, commandes exécutées, résultats ST-SEC* et ST-F*, limites connues. Il sert de preuve complémentaire au plan 1.3.7 et aux scénarios 1.2.2.

Ce qui a été mis en place

Environnements utilisés

ENVIRONNEMENT	URL	USAGE
QA local	http://127.0.0.1:4173	ST-F01 à ST-F06 (Playwright)
PROD	https://dirmhublot.netlify.app	ST-SEC01, ST-F01 (partie publique), ST-F06
STAGING	staging--dirmhublot.netlify.app	404 — branch deploy à activer ; non utilisé

Commandes exécutées

TERMINAL
npm run security:headers npm run security:test npm run test:campaign

Fichiers Playwright : e2e/campaign-manual.spec.ts , e2e/campaign-prod.spec.ts . **Résultat Playwright** : 9 passed (6 scénarios QA + 3 vérifications PROD).

Preuves et démonstration

ST-SEC01 — Headers (PROD)

Commande : curl -sI https://dirmhublot.netlify.app ou npm run security:headers

CRITÈRE	RÉSULTAT
HTTPS / HSTS	strict-transport-security présent
X-Frame-Options	DENY
X-Content-Type-Options	nosniff
Statut	HTTP/2 200

Résultat : Passé

ST-SEC02 — Garde-fous applicatifs

Commande : npm run security:test

CRITÈRE	RÉSULTAT
Tests exécutés	15
Sanitization anti-XSS	Passé
Validation filtres	Passé
Session / expiration 8 h	Passé

Résultat : Passé**Synthèse scénarios fonctionnels**

ID	RÉSULTAT	PREUVE
ST-F01	Passé	Playwright : refus login, connexion, déconnexion (QA) ; PROD : page login sans session
ST-F02	Passé	3 onglets, pas d'erreur console bloquante
ST-F03	Passé	Filtre région + réinitialisation
ST-F04	Passé	Viewport 375px, <code>main</code> sans débordement majeur
ST-F05	Passé	3 onglets en moins de 15 s
ST-F06	Passé	QA : badge « Test local (QA) » ; PROD : aucun badge
ST-AUTO-*	Passé	CI GitHub — 1.3.7

Limites de la campagne

- Identifiants **PROD** Netlify non testés manuellement (secrets hors dépôt) ; login complet validé en **QA**.
- **STAGING** : à rejouer dès branch deploy active.
- Données RH réelles : campagne QA sur jeu embarqué / mock ; validation données réelles = STAGING interne.

Prochaines actions

- 1 Activer branch deploy **staging** sur Netlify.
- 2 Rejouer **ST-F01** à **F03** sur staging avec identifiants staging.
- 3 Conserver captures Actions + ce rapport pour la traçabilité.

SYNTHÈSE

La campagne du 27 mai 2026 valide les scénarios ST-F* en QA et ST-SEC01/02 en prod/local, avec 9 tests Playwright passés et CI alignée sur 48 tests Vitest.

1.2.10 Guide Playwright**EN BREF**

Playwright pilote un navigateur réel pour les tests **E2E** de Hublot : login, tableau de bord, campagne ST-F*. En CI, les tests tournent **headless** ; en démo, `npm run test:e2e:demo` af-

fiche le parcours en ralenti. Vitest couvre les **48 tests unitaires** sans navigateur. Fichiers : `playwright.config.ts`, dossier `e2e/`.

Ce qui a été mis en place

COMMANDE	FENÊTRE	USAGE
<code>npm run test:e2e</code>	Non (headless)	CI, vérif locale
<code>npm run test:e2e:demo</code>	Oui, ~800 ms	Démonstration
<code>npm run test:e2e:slow</code>	Oui, ~400 ms	Démo plus vive
<code>npm run test:e2e:headed</code>	Oui, rapide	Debug visuel
<code>npm run test:e2e:ui</code>	UI Playwright	Un test à la fois, replay
<code>npm run test:campaign</code>	Headless	ST-F01... + prod lecture seule

Fichiers :

`e2e/` → `smoke.spec.ts` # 3 tests — parcours critique → `campaign-manual.spec.ts` # ST-F01...F06
→ `campaign-prod.spec.ts` # v...

Scénario `smoke.spec.ts`

#	TEST	VÉRIFICATION
1	Page de connexion	« Bienvenue », champs, bouton Se connecter
2	Connexion → dashboard	<code>e2e-user</code> / <code>e2e-pass</code> , bouton Déconnexion
3	<code>health.json</code>	<code>{ "status": "ok", "app": "hublot" }</code>

Identifiants **uniquement** pour `build:e2e`, pas les secrets Netlify prod.

Preuves et démonstration

TERMINAL		
<pre>npm ci npx playwright install chromium npm run test:e2e:demo ## Attendu : 3 passed ## ... (voir dépôt)</pre>		
LOCAL DÉMO		GITHUB ACTIONS
Fenêtre	Visible (<code>test:e2e:demo</code>)	Headless
URL	127.0.0.1:4173	Idem sur runner
Preuve	Terminal + <code>playwright-report/</code>	Job E2E vert

Pièges fréquents

PROBLÈME	SOLUTION
Trop rapide	<code>test:e2e:demo</code> ou <code>PLAYWRIGHT_SLOW_MS=1200</code>
<code>--debug</code> + page blanche	Cliquer Resume ; préférer demo ou UI
Premier lancement long	Attendre <code>build:e2e</code> (~20 s)
Port 4173 occupé	<code>lsof -i :4173</code> , fermer autre preview
Tester vraie prod login	Utiliser <code>campaign-prod.spec.ts</code> (lecture seule)

Checklist avant livraison

- `npm ci` + `playwright install chromium`
- `npm run test:e2e:demo` → 3 passed
- Expliquer Vitest vs Playwright
- Onglet Actions CI (job E2E) prêt

SYNTHÈSE

Playwright verrouille le parcours utilisateur critique de Hublot en local et en CI ; la démonstration utilise `test:e2e:demo`, la campagne documentée `test:campaign` pour les scénarios ST-F*.

Rédiger des scripts dans la démarche DevOps

1.3.1 Les bases du déploiement automatique

EN BREF

Ce chapitre pose les fondations du **déploiement automatique** pour Hublot : scripts reproductibles, séparation CI (qualité) et CD (publication), deux cibles (Netlify cloud et NAS Synology interne). Nous avons choisi des scripts Bash idempotents et des workflows GitHub Actions plutôt qu'une exécution manuelle à chaque livraison. Le dépôt `Meriem1403/Hublot` et la production `https://dirmhublot.netlify.app` servent de référence concrète.

Ce qui a été mis en place

Nous avons structuré le déploiement en **trois couches** :

1. **Qualité automatisée** — workflow GitHub **CI** : ESLint, `npm run test:run` (48 tests), `npm run audit:prod`, build Vite, tests Playwright E2E.
2. **CD cloud** — Netlify relie le dépôt `main` : build `npm run build`, publication du dossier `build/` vers `https://dirmhublot.netlify.app` (détail dans 1.1.5.md)).
3. **CD interne** — script `scripts/deploy-nas.sh` : build local contrôlé, copie `build/` et `dist/`, rsync SSH optionnel vers un Synology.

Les scripts vivent dans `scripts/` ; la documentation opérationnelle est dans `scripts/README.md` et 1.2.7.

Preuves et démonstration

Après un push sur `main`, vérifier que le workflow **CI** est vert sur GitHub Actions — Meriem1403/Hublot (<https://github.com/Meriem1403/Hublot/actions/workflows/ci.yml>).

TERMINAL

```
## Reproduire localement la gate qualité du déploiement
npm ci
npm run lint
npm run test:run # attendu : 48 tests passés
npm run build
test -f build/index.html

> Pour le NAS, sans rsync :
bash
./scripts/deploy-nas.sh
## Artefacts prêts : build/ et dist/
```

ÉLÉMENT	VALEUR ATTENDUE
Tests unitaires	48 passed (Vitest)
Prod cloud	<code>https://dirmhublot.netlify.app</code>
Workflow CI	<code>.github/workflows/ci.yml</code>

SYNTHÈSE

Le déploiement automatique de Hublot repose sur la CI GitHub (qualité), la CD Netlify (prod publique) et le script `deploy-nas.sh` (réseau interne), tous reproductibles et alignés sur les 48 tests Vitest.

1.3.2 Rédiger et utiliser un script de déploiement

EN BREF

Ce chapitre décrit le script `scripts/deploy-nas.sh`, qui prépare et livre Hublot vers un NAS Synology : installation des dépendances, contrôles qualité (lint + 48 tests Vitest), build de production, puis copie locale ou synchronisation SSH. Il complète la CD Netlify pour un usage en réseau interne DIRM, sans exécuter `npm ci` sur le NAS.

Ce qui a été mis en place

Le fichier `scripts/deploy-nas.sh` (Bash, ~50 lignes) réalise :

1. `npm ci` — dépendances figées (`package-lock.json`).
2. `npm run lint` — ESLint.
3. `npm run test:run` — **48 tests** Vitest.
4. `npm run build` — compilation production.
5. Vérification de `build/index.html` .
6. Miroir `build/` → `dist/` (certains `docker-compose` montent `dist/`).
7. Mode par défaut : message pour copie manuelle du projet sur le NAS.
8. Mode `--rsync` : envoi vers `NAS_USER@NAS_HOST:NAS_PATH` si les trois variables sont définies.

Fichiers Docker associés : `docker-compose.yml` (Nginx + volume statique, port **8080**), `nginx.conf` , variante `docker-compose.prod.yml` pour build sur NAS (non recommandée si Container Manager bloque sur `npm ci`).

Preuves et démonstration

TERMINAL

```
./scripts/deploy-nas.sh
## Attendu en fin de log :
## ==> Artefacts prêts : build/ et dist/

npm run test:run
## ... (voir dépôt)
```

Checklist opérationnelle :

- Conteneur **Running** dans Container Manager.
- `build/index.html` présent sur le NAS.
- Refresh sur une route React (SPA → `index.html` via `nginx.conf`).
- Données : fichier `public/data/agents.json` déployé ou volume à jour.

Référence racine (copie courte) : `DEPLOY_SYNOLOGY_LOCAL.md` .

SYNTHÈSE

`deploy-nas.sh` formalise le déploiement interne de Hublot : mêmes garde-fous que la CI, artefacts `build/` / `dist/` , publication manuelle ou rsync vers le Synology.

1.3.3 Les bases des scripts d'évolution

EN BREF

Les **scripts d'évolution** mettent à jour les données RH de Hublot à partir des exports Excel RenoïRH (feuille ETPT), sans modifier le code React à chaque mouvement d'effectifs. Nous avons défini une chaîne traçable : conversion Python, contrôles de cohérence, publication optionnelle vers Neon. Le point d'entrée standard est `scripts/run-evolution-pipeline.sh`, appuyé sur `convert_suivi_etpt_to_json.py`.

Ce qui a été mis en place

COMPO-SANT	FICHER	RÔLE
Orchestrateur	<code>scripts/run-evolution-pipeline.sh</code>	Enchaîne conversion, contrôles Python inline, SHA256
Convertisseur	<code>scripts/convert_suivi_etpt_to_json.py</code>	Excel « Données annuelles » → JSON StatDirm
Sorties	<code>src/data/agents.json</code> , <code>public/data/agents.json</code>	Build + servage HTTP
Legacy	<code>scripts/convert_excel_to_json.py</code>	Ancien format Interface_Effectifs (référence)
Diagnostic	<code>scripts/analyze_excel.py</code>	Structure des colonnes Excel
Application	<code>src/services/dataService.ts</code> , <code>src/hooks/useAgentsData.ts</code>	Chargement et agrégations

Le modèle cible des champs est documenté dans 1.3.3.1. La rédaction détaillée du pipeline : 1.3.4.

Preuves et démonstration

TERMINAL

```
bash scripts/run-evolution-pipeline.sh --file "trdata/Suivi_des_emplois_en_ETPT_RPROG (23).xlsx"
## Logs attendus : Agents totaux / Agents actifs / Contrôles : OK
## SHA256 agents.json: <empreinte>

npm run test:run
## 48 passed — non-régression métier après nouvelles données
```

Vérification manuelle rapide : ouvrir Hublot, filtre **DIRM Méditerranée** (ou région cible), contrôler effectifs et graphiques missions.

SYNTHÈSE

Les scripts d'évolution de Hublot transforment l'Excel ETPT en JSON validé, via `run-evolution-pipeline.sh` et `convert_suivi_etpt_to_json.py`, avant toute mise en ligne Netlify ou NAS.

1.3.3.1 Modèle de données cible

EN BREF

Ce chapitre décrit le **modèle de données cible** de Hublot : structure JSON `agents.json`, interface TypeScript `Agent` dans `src/types/data.ts`, champs issus de l'export ETPT et agrégats calculés par l'application (ETP, répartitions, filtres PASA). Il sert de contrat entre les scripts d'évolution et le front React.

Ce qui a été mis en place

Fichier de référence TypeScript

Le contrat applicatif est dans `src/types/data.ts` : interface `Agent` (identité, affectation, temps de travail, métadonnées) et types dérivés (`OverviewStats`, répartitions âge/genre/statut, capacités missions/régions).

Champs notables ajoutés pour l'export RenoïRH :

- `uniteService`, `missionCode` — granularité opérationnelle.
- `pasaCode`, `pasaLibelle`, `pasaSegment`, `pasaSousSegment` — filtres politiques publiques.
- `corps`, `fonctionExercee`, `fonctionCategorie` — référentiels grade/poste.
- `tempsTravailRenseigne` — traçabilité colonne source.

Enveloppe JSON

JSON

```
{
  "agents": [ { "id": "...", "nom": "...", ... } ],
  "metadonnees": {
    "dateExport": "YYYY-MM-DD",
    "source": "Suivi ETPT_RPROG",
    "version": "1.0"
  }
}
```

> Les fichiers générés sont écrits en parallèle dans `src/data/agents.json` et `public/data/agents.json`. → **### Champs**

...

bash

Générer les données conformes au modèle

bash scripts/run-evolution-pipeline.sh --file "trdata/Suivi_des_emplois_en_ETPT_RPROG (23).xlsx"

Vérifier structure minimale

python3 -c "import json; d=json.load(open('public/data/agents.json')); a=d['agents'][0]; print(sorted(a.keys())[:12])"

... (voir dépôt)

Contrôle manuel : dans l'app, filtrer une région et un service — les compteurs doivent correspondre à un sous-ensemble cohérent du JSON.

SYNTHÈSE

Le modèle cible Hublot est l'interface `Agent` et l'enveloppe `agents.json`, alimentés par le convertisseur ETPT et validés par le pipeline d'évolution et les 48 tests Vitest.

1.3.4 Rédiger des scripts d'évolution

EN BREF

Ce chapitre détaille la **rédaction** et l'**exécution** des scripts d'évolution de Hublot : orchestrateur Bash `run-evolution-pipeline.sh`, convertisseur Python `convert_suivi_etpt_to_json.py`, contrôles intégrés et option `--push-neon`. L'objectif est un script lisible, paramétrable et sûr avant toute publication des données RH.

Ce qui a été mis en place

`run-evolution-pipeline.sh`

- Options : `--file`, `--push-neon`, `--help`.
- `set -euo pipefail` — arrêt immédiat si une étape échoue.
- Mesure du temps par étape (`step_start` / `step_end`).
- Étapes : conversion → contrôles JSON → SHA256 → push Neon optionnel.

`convert_suivi_etpt_to_json.py`

- Lecture feuille `Données annuelles`, en-tête ligne 5 (`HEADER_ROW = 4`).
- Dictionnaire `COL` : libellés Excel exacts RenoiRH.
- Réduction **un agent par matricule** (dernière observation retenue).
- Dérivation **genre / dateNaissance** depuis le NIR si absentes.
- Écriture `src/data/agents.json` et `public/data/agents.json`.

Scripts npm liés

COMMANDE	RÔLE
<code>npm run copy-data-for-netlify</code>	Copie <code>src/data</code> → <code>public/data</code> si besoin
<code>npm run push-agents-to-neon</code>	Publication base Neon (option 2 Netlify)

Index opérationnel : `scripts/README.md`.

Preuves et démonstration

Sortie attendue (extrait) :

==> Conversion Excel -> JSON → ... → ==> Contrôles de cohérence JSON → Agents totaux : → Agents actifs : → # ...

TERMINAL

```
npm run test:run
## 48 passed

grep -c '"id"' public/data/agents.json
## cohérent avec le log « Agents totaux »
```

SYNTHÈSE

Les scripts d'évolution de Hublot sont écrits pour être sûrs par défaut : `convert_suivi_etpt_to_json.py` produit le JSON, `run-evolution-pipeline.sh` valide et trace, `--push-neon` ne s'exécute qu'à la demande.

1.3.4.1 Publier et vérifier les données sur Netlify

EN BREF

Ce chapitre explique **comment publier** les données RH de Hublot sur **Netlify** et **vérifier** qu'elles s'affichent sur `https://dirmhublot.netlify.app`. Deux modes existent : JSON statique dans le dépôt (recommandé par défaut) ou base **Neon** via `NETLIFY_DATABASE_URL`. Dans les deux cas, la chaîne d'évolution (`run-evolution-pipeline.sh`, `convert_suivi_etpt_to_json.py`) précède la publication.

Ce qui a été mis en place

ÉLÉMENT	FICHER / COMMANDE
Conversion	<code>convert_suivi_etpt_to_json.py</code> → <code>public/data/agents.json</code>
Orchestration	<code>run-evolution-pipeline.sh --file ...</code>
Copie de secours	<code>npm run copy-data-for-netlify</code> (<code>scripts/copy-data-for-netlify.js</code>)
Push base	<code>npm run push-agents-to-neon</code> (<code>scripts/push-agents-to-neon.js</code>)
CD	Netlify build <code>npm run build</code> , publish <code>build/</code>
Qualité amont	<code>.github/workflows/ci.yml</code> — 48 tests avant merge

Preuves et démonstration

TERMINAL

```
## Local
bash scripts/run-evolution-pipeline.sh --file "trdata/Suivi_des_emplois_en_ETPT_RPROG (23).xlsx"
npm run test:run # 48 passed

## Après push main – URL prod
curl -sI https://dirmhublot.netlify.app/data/agents.json | head -5
```

Capture attendue pour la traçabilité : dashboard Hublot avec graphiques alimentés, deploy Netlify Published sur le SHA du commit données, workflow **CI** vert sur le même SHA.

SYNTHÈSE

Publier les données Hublot sur Netlify, c'est d'abord générer un `agents.json` validé par le pipeline d'évolution, puis le committer (option 1) ou le pousser vers Neon (option 2), et contrôler l'URL de production après CD.

1.3.5 Optimiser les scripts d'évolution

EN BREF

Ce chapitre présente les **optimisations** apportées aux scripts d'évolution de Hublot : mesure du temps par étape, mode dry-run par défaut, point de publication explicite (`--push-neon`), contrôles automatiques et empreinte SHA256. L'objectif est de réduire les erreurs humaines et d'accélérer le diagnostic quand un export Excel change de structure.

Ce qui a été mis en place

OPTIMISATION	OÙ	BÉNÉFICE
Chronométrage	<code>run-evolution-pipeline.sh</code>	Repérer une conversion lente ou bloquante
Dry-run par défaut	Pas de <code>--push-neon</code>	Pas de publication DB accidentelle
Publication explicite	Flag <code>--push-neon</code>	Séparation validation / mise en prod données
Contrôles structurels	Bloc Python inline	IDs, actifs, services — avant commit
Empreinte SHA256	<code>shasum -a 256</code> sur <code>public/data/agents.json</code>	Preuve lecteur (« quelle version est en prod ? »)
Déduplication matricule	<code>convert_suivi_etpt_to_json.py</code>	Fichier JSON plus petit, chargement UI plus rapide
Double sortie contrôlée	<code>src/data</code> + <code>public/data</code>	Pas d'écart build Netlify / dev

Pistes non prioritaires (si volume ×10) : cache pandas intermédiaire, parallélisation par feuille — non nécessaires au périmètre actuel DIRM.

Preuves et démonstration

TERMINAL

```
bash scripts/run-evolution-pipeline.sh --file "trdata/Suivi_des_emplois_en_ETPT_RPROG (23).xlsx" 2>&1 | tee /tmp/pipeline.log
grep -E '^(==>|SHA256|Agents|Contrôles)' /tmp/pipeline.log

npm run test:run
## 48 passed — garde-fou après optimisation données
```

Pour le lecteur : joindre log pipeline (timings + SHA256) + capture CI verte + URL prod avec effectifs à jour.

SYNTHÈSE

Les scripts d'évolution Hublot sont optimisés pour la sûreté et la traçabilité : temps mesuré, dry-run par défaut, validations et empreinte SHA256 avant toute publication Netlify ou Neon.

1.3.6 Ecrire un script YAML d'Intégration Continue

EN BREF

Ce chapitre explique comment le workflow **CI** de Hublot est écrit en **YAML** (`.github/workflows/ci.yml`) : déclencheurs, jobs parallèles, dépendances `needs`, artefact de build et enchaînement E2E. La CI exécute lint, **48 tests** Vitest, audit npm et build avant toute CD Netlify ou NAS.

Ce qui a été mis en place

Fichier `.github/workflows/ci.yml` (commentaire d'en-tête : CI sans déploiement ; CD = Netlify + `deploy-nas.sh`).

JOB	COMMANDE CLÉ	RÔLE
<code>lint</code>	<code>npm run lint</code>	Qualité code ESLint
<code>unit-tests</code>	<code>npm run test:run</code>	48 tests Vitest
<code>audit</code>	<code>npm run audit:prod</code>	Vulnérabilités dépendances prod
<code>build</code>	<code>npm run build</code> + vérif <code>build/index.html</code>	Artefact Vite
<code>e2e</code>	<code>npm run test:e2e</code> (Playwright)	Parcours login / dashboard

Paramètres transverses :

- **on** : `push` et `pull_request` sur `main`, `staging` ; `workflow_dispatch`.
- **concurrency** : `ci-${{ github.ref }}`, `cancel-in-progress: true`.
- **cache** : `cache: npm` dans `setup-node@v4`.
- **artefact** : `upload build/` (`retention-days: 7`).

CD séparée : `cd-netlify.yml`, `cd-nas.yml` (voir 1.1.5.md)).

Preuves et démonstration

- Onglet Actions : workflow **CI** vert sur le SHA de `main`.
- Job **Tests unitaires** : log `48 passed`.
- Job **Build production** : présence artefact `build-<sha>`.
- Job **Tests E2E** : scénarios Playwright passés.

TERMINAL

```
## Local – reproduire la gate
npm run test:run
## Tests 48 passed (4 files)
```

Fichier source : `.github/workflows/ci.yml` à la racine du dépôt.

SYNTHÈSE

Le YAML `ci.yml` formalise l'intégration continue de Hublot : parallélisme des contrôles, build artefact, E2E en aval, sans mélanger le déploiement.

1.3.7 Automatiser les tests en DevOps

EN BREF

Ce chapitre décrit l'**automatisation des tests** de Hublot dans la démarche DevOps : **48 tests** unitaires Vitest, tests de sécurité, audit npm, build et E2E Playwright, tous déclenchés par `.github/workflows/ci.yml` à chaque push ou pull request. Les scripts `deploy-nas.sh` et le pipeline d'évolution s'appuient sur la même commande `npm run test:run` pour éviter de livrer une régression.

Ce qui a été mis en place

Batterie unitaire (48 tests)

FICHIER	TESTS	COUVERTURE
<code>src/services/dataService.test.ts</code>	12	Filtres, chargement, normalisation
<code>src/utils/dataCalculations.test.ts</code>	20	Âge, ETP, répartitions, stats service
<code>src/utils/security.test.ts</code>	15	Sanitization, session, masquage RH
<code>src/config/environment.test.ts</code>	1	Libellé environnement DEV / prod

Commande : `npm run test:run`.

Autres contrôles automatisés

OUTIL	COMMANDE	OÙ
ESLint	<code>npm run lint</code>	Job <code>lint</code> dans <code>ci.yml</code>
Audit prod	<code>npm run audit:prod</code>	Job <code>audit</code>
Build	<code>npm run build</code>	Job <code>build</code>
Playwright	<code>npm run test:e2e</code>	Job <code>e2e</code> (après build)

Intégration dans les scripts DevOps

- `ci.yml` — exécute toute la chaîne sur GitHub Actions.
- `deploy-nas.sh` — `npm run test:run` avant `npm run build`.
- **Évolution données** — après `run-evolution-pipeline.sh`, `npm run test:run` recommandé avant commit.

Plan et scénarios manuels complémentaires : 1.2.5, 1.2.8.md).

Preuves et démonstration

TERMINAL

```
npm run test:run

> Sortie attendue (ordre de grandeur) :

Test Files 4 passed (4)
Tests 48 passed (48)
```

Sur GitHub : workflow **CI** → job **Tests unitaires** → log vert.

PREUVE	OÙ LA TROUVER
48 passed	Sortie <code>npm run test:run</code> ou log Actions
E2E	Job <code>e2e</code> dans <code>ci.yml</code>
Prod stable	<code>https://dirmhublot.netlify.app</code> + dernier SHA CI vert
Rapport d'exécution	1.2.9

TERMINAL

```
## Sécurité (inclus dans les 48 via security.test.ts)
npm run test:run -- src/utils/security.test.ts
```

SYNTHÈSE

Hublot automatise **48 tests** Vitest et la chaîne lint / audit / build / E2E dans `ci.yml`, avec la même gate dans `deploy-nas.sh` et après chaque évolution de données.